Berkeley Engineering | BerkeleyHaas

PROFESSIONAL CERTIFICATE IN MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Let’s give everyone a couple of minutes to join…

Module 8

Feature Engineering ML & LLM

Toby Gardner

5 November 2025

Feature Engineering

Feature Engineering in ML and LLMs

- In our last session, we expanded on the curriculum by looking deeper on how to apply unsupervised learning techniques such as DBScan Clustering and PCA not only for EDA and dimensionality reduction but also on how we can create new features aka feature engineering.
- If you're interested in reviewing feature engineering for numerical data (with examples), check out previous slides and notebooks

- In this session, we're going to explore Feature Engineering for categorical features and text-based data: preparing you not only in traditional ML but also for Generative AI such as LLMs.

Meet your Learning Facilitator!

Hi everyone, I'm Toby 👋

- MBA (Masters in Business Admin.) from the University of California at Berkeley CA, USA.
- MAIDS (Masters of Applied Data Science) from the University of Michigan, MI, USA
- CFA Charterholder

- Co-founder Finch: financial SAAS using machine-learning to provide data-intelligence
- Prior to Finch, Toby was a Director at KPMG - where he was the top rated consultant in Australia

This is what Midjourney says I should look like (using my bio as a prompt)

Github practise! https://github.com/toby-gardner-ai

Office Hour Expectations

My objective: to get you to think like a real data scientist

- Understand at what level you will need to grasp AI ML concepts in practise ✅
- Understand when to *think* hard and move slow ✅
- Understand when to *move quickly* and break things ✅
- Foster a culture of collaboration and knowledge-sharing ✅
- Discuss ~~theoretical~~ real-world data and challenges ✅
- Discrepancies between course content ❌
- I can't open a notebook ❌
- Library not working ❌

Section 1

Encoding

```python
sentence = "I love everything involving AI and machine learning and love the Universit

words = sentence.split()

# dict.fromkeys() --> keeps order and the first time each word appears
unique_words = list(dict.fromkeys(words))

df = pd.DataFrame(words, columns=['word'])
```

✓ 0.0s Python

Ordinal Encoding

IF you have categorical data WITH inherent order e.g., small/medium/large

```
1  ord_encoder = OrdinalEncoder(categories=[unique_words],dtype=int)
2
3  encoded_ord = ord_encoder.fit_transform(df[['word']].astype(str))
4
5  df_ordinal = df.copy()
6  df_ordinal['ordinal_id'] = encoded_ord.astype(int)
7
8  df_ordinal
```

✓ 0.0s Python

	word	ordinal_id
0	I	0
1	love	1
2	everything	2
3	involving	3
4	AI	4
5	and	5
6	machine	6
7	learning	7
8	and	5

One Hot Encoding

IF you have categorical data with NO inherent order (e.g., colors, countries

```
1  oh_encoder = OneHotEncoder(categories=[unique_words], sparse_output=False, dtype=int,
2
3  encoded_oh = oh_encoder.fit_transform(df[['word']])
4
5  encoded_df = pd.DataFrame(encoded_oh, columns=[f'_{word.lower()}' for word in unique_w
6
7  df_one_hot = pd.concat([df.reset_index(drop=True), encoded_df], axis=1)
8  df_one_hot
9
```

✓ 0.0s Python

	word	_i	_love	_everything	_involving	_ai	_and	_machine	_learning	_the	_univer
0	I	1	0	0	0	0	0	0	0	0	
1	love	0	1	0	0	0	0	0	0	0	
2	everything	0	0	1	0	0	0	0	0	0	
3	involving	0	0	0	1	0	0	0	0	0	
4	AI	0	0	0	0	1	0	0	0	0	
5	and	0	0	0	0	0	1	0	0	0	
6	machine	0	0	0	0	0	0	1	0	0	
7	learning	0	0	0	0	0	0	0	1	0	

Advanced One Hot Encoding

From Text → Categorical Data → One Hot Encoding

```
1  categorise_text = spacy.load("en_core_web_sm")
2
3  text = categorise_text(sentence)
4
5  words = []
6  pos_categ = []
7
8  for token in text:
9      if token.is_space or token.is_punct:
10         continue
11     words.append(token.text)
12     pos_categ.append(token.pos_)
13
14 df_pos = pd.DataFrame({'word': words,'pos': pos_categ})
15
16 unique_pos = list(dict.fromkeys(pos_categ))
17
18 oh_encoder = OneHotEncoder(categories=[unique_pos], sparse_output=False, dtype=int, ha
19
20 encoded_oh = oh_encoder.fit_transform(df_pos[['pos']])
21
22 encoded_df = pd.DataFrame(encoded_oh, columns=[f'_{pos.lower()}' for pos in unique_pos
23
24 df_pos_ohe = pd.concat([df_pos.reset_index(drop=True), encoded_df], axis=1)
25 df_pos_ohe
26
```

✓ 0.1s Python

	word	pos	_pron	_verb	_propn	_cconj	_noun	_det	_adp
0	I	PRON	1	0	0	0	0	0	0
1	love	VERB	0	1	0	0	0	0	0
2	everything	PRON	1	0	0	0	0	0	0
3	involving	VERB	0	1	0	0	0	0	0

LLMs: these ain't your granny's ML

LLMs encode data not 'features'

- In traditional ML, we encode text features using techniques such as ordinal encoding or one-hot encoding to transform the features into a numerical format the model can ingest.
- These encoders give numerical structure to non-numeric features. However, they rely entirely on human-defined categories.
- In contrast, LLMs encode before features have been identified

Why?

- Because LLMs identify features themselves implicitly during model training, not explicitly before training
  - ML Features: explicit, identified by the Data Scientist before training
  - LLM Features: implicit, discovered by the Model during training

LLMs: these ain't your granny's ML

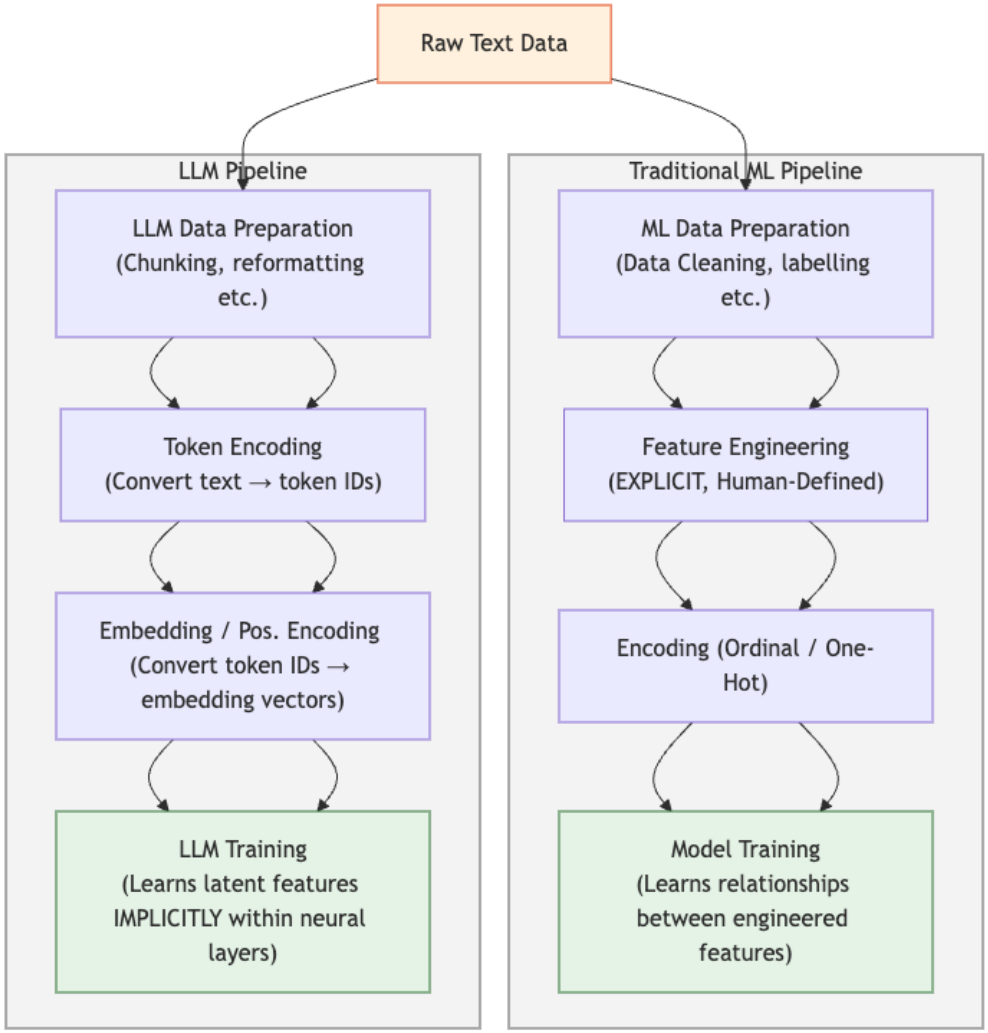
LLMs do a lot of the encoding and feature 'engineering' themselves

- In traditional ML, we encode text features using techniques such as ordinal encoding or one-hot encoding to transform the features into a numerical format the model can ingest.
- These encoders give numerical structure to non-numeric features. However, they rely entirely on human-defined categories.
- In contrast, LLMs encode before features have been identified.

Why?

- Because LLMs identify features implicitly during model training, not explicitly before training.
  - ML Features: explicit, identified by the Data Scientist before training
  - LLM Features: implicit, discovered by the Model during training

Visual Comparison

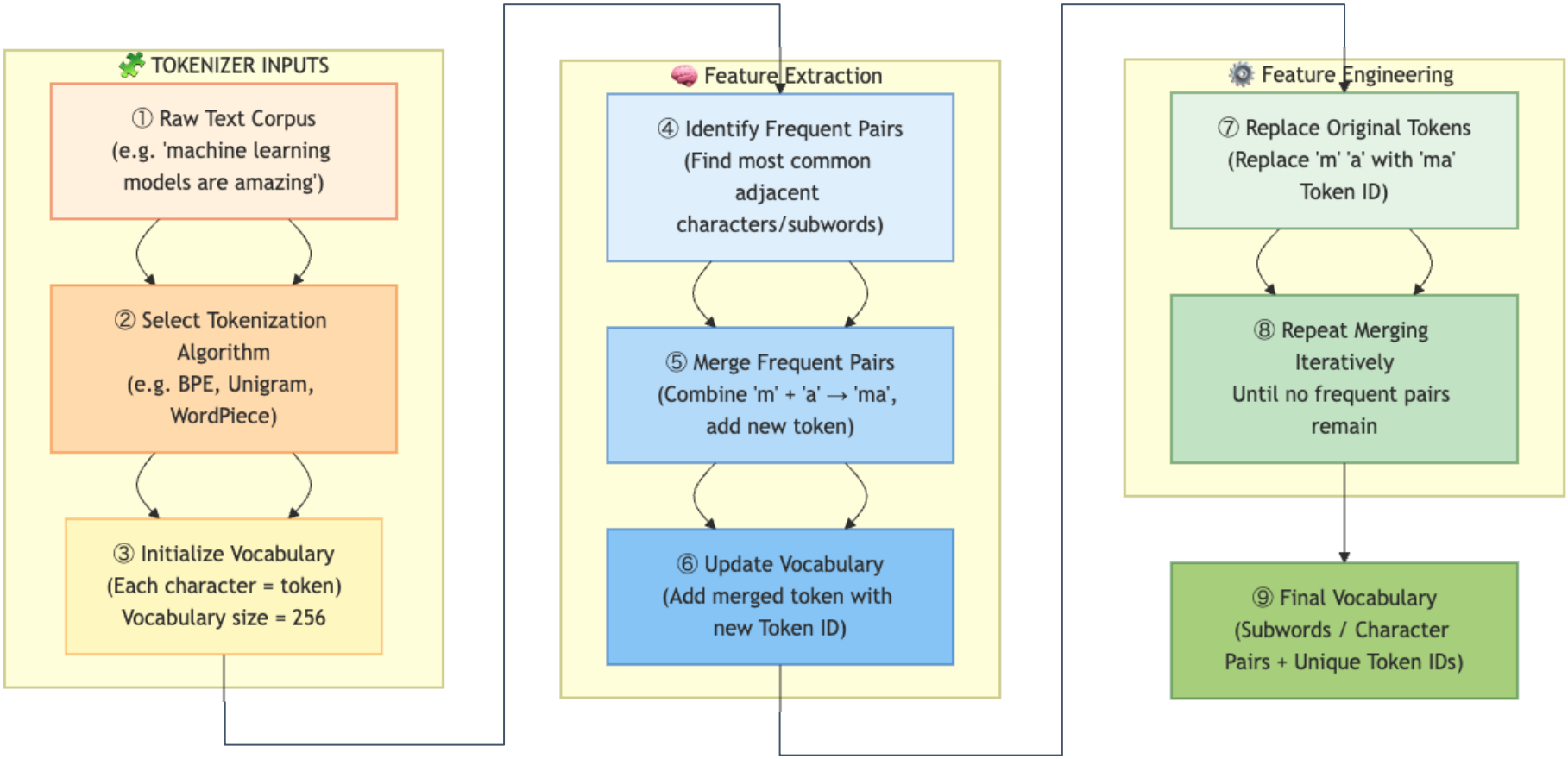


LLM Encoding

First, break down the text data into small, unique strings we can encode

- These small, unique strings are frequently appearing character combinations in languages - often representing words or sub-words. Once identified, these can be mapped to a unique ID and subsequently encoded in the text by replacing the string with the ID.
- In practise, this encoding process is so time-consuming and complicated (how can you guarantee the sub-string and ID matches don't change when you update your training corpus?!?!) that it requires a model of its own.
- As a result, most LLMS use pre-trained encoders created from other models.
- These models are referred to as 'Tokenizers'.

Tokenizers


TOKENIZER INPUTS
① Raw Text Corpus (e.g. 'machine learning models are amazing')
② Select Tokenization Algorithm (e.g. BPE, Unigram, WordPiece)
③ Initialize Vocabulary (Each character = token) Vocabulary size = 256
Feature Extraction
④ Identify Frequent Pairs (Find most common adjacent characters/subwords)
⑤ Merge Frequent Pairs (Combine 'm' + 'a' → 'ma', add new token)
⑥ Update Vocabulary (Add merged token with new Token ID)
Feature Engineering
⑦ Replace Original Tokens (Replace 'm' 'a' with 'ma' Token ID)
⑧ Repeat Merging Iteratively Until no frequent pairs remain
⑨ Final Vocabulary (Subwords / Character Pairs + Unique Token IDs)

Encoding comparisons

Tokenization can encode words it's never seen before

	word	token_id	_pron	_verb	_propn	_cconj	_noun	_det	_adp	ordinal_id
0	I	[39]	1	0	0	0	0	0	0	0
1	love	[137, 114]	0	1	0	0	0	0	0	1
2	everything	[1024, 79, 86, 98]	1	0	0	0	0	0	0	2
3	involving	[83, 76, 722]	0	1	0	0	0	0	0	3
4	AI	[134]	0	0	1	0	0	0	0	4
5	and	[96]	0	0	0	1	0	0	0	5
6	machine	[750]	0	0	0	0	1	0	0	6
7	learning	[448]	0	0	0	0	1	0	0	7
8	and	[96]	0	0	0	1	0	0	0	5
9	love	[137, 114]	0	1	0	0	0	0	0	1
10	the	[99]	0	0	0	0	0	1	0	8
11	University	[842]	0	0	1	0	0	0	0	9
12	of	[103]	0	0	0	0	0	0	1	10

In our original sentence, the word "involving" (idx: 3) is tokenized BUT doesn't exist in the corpus!

Instead, the string:
- "inv" appeared 8 times in the corpus from the word "invite" --> encoded with token id: 83
- "olv" appeared 6 times in the corpus, from words such as "solve" and "evolve" --> encoded with token id: 76
- "ing" appeared 152 times in the corpus, from lots of words! --> encoded with token id 722

Takeaway

Data preparation and encoding still applies

- whether you're engineering one-hot vectors or training massive LLMs, both rely on feature encoding — the key difference lies in who engineers the features: you, or the model.

QUESTIONS?